

memory_monitor.py

```
import psutil
import os
import gc
from datetime import datetime
import traceback

class MemoryMonitor:
    """Monitor and manage memory usage - CRITICAL for Render free tier"""

    def __init__(self, max_memory_mb=400):
        self.max_memory_mb = max_memory_mb
        self.process = psutil.Process(os.getpid())
        self.is_render = os.environ.get('RENDER') == 'true'
        self.warnings_issued = 0

    def get_memory_usage(self):
        """Get current memory usage in MB"""
        try:
            memory_info = self.process.memory_info()
            memory_mb = memory_info.rss / 1024 / 1024 # Convert to MB
            return memory_mb
        except Exception as e:
            print(f"[MemoryMonitor] Error getting memory: {e}")
            return 0

    def get_memory_percent(self):
        """Get memory usage as percentage of limit"""
        current = self.get_memory_usage()
        return (current / self.max_memory_mb) * 100

    def log_memory(self, context=""):
        """Log current memory usage"""
        memory_mb = self.get_memory_usage()
        percent = self.get_memory_percent()

        status = "?"
        if percent > 90:
            status = "? CRITICAL"
        elif percent > 75:
            status = "?? WARNING"
        elif percent > 60:
            status = "?"

        message = f"[Memory {status}] {context}: {memory_mb:.1f}MB / {self.max_memory_mb}MB ({percent:.1f}%)\"
        print(message)

        return {
            'memory_mb': memory_mb,
            'percent': percent,
            'max_mb': self.max_memory_mb,
            'status': status
        }

    def check_memory_limit(self, context=""):
        """Check if approaching memory limit and take action"""
        memory_mb = self.get_memory_usage()
        percent = self.get_memory_percent()

        # Critical: Over 90%
        if percent > 90:
            print(f"? [Memory CRITICAL] {context}: {memory_mb:.1f}MB ({percent:.1f}%)")
            self.force_garbage_collection()
            return False # Signal to stop processing

        # Warning: Over 75%
        elif percent > 75:
            print(f"?? [Memory WARNING] {context}: {memory_mb:.1f}MB ({percent:.1f}%)")
            self.warnings_issued += 1
            if self.warnings_issued > 3:
                self.force_garbage_collection()
            self.warnings_issued = 0
```

```

return True # Can continue but be careful

return True # All good

def force_garbage_collection(self):
    """Force garbage collection to free memory"""
    print("[Memory] ? Running garbage collection...")
    before = self.get_memory_usage()

    gc.collect()

    after = self.get_memory_usage()
    freed = before - after

    print(f"[Memory] ? Freed {freed:.1f}MB (was {before:.1f}MB, now {after:.1f}MB)")

    return freed

def get_stats(self):
    """Get comprehensive memory statistics"""
    memory_mb = self.get_memory_usage()
    percent = self.get_memory_percent()

    return {
        'current_mb': round(memory_mb, 2),
        'max_mb': self.max_memory_mb,
        'percent_used': round(percent, 2),
        'available_mb': round(self.max_memory_mb - memory_mb, 2),
        'is_render': self.is_render,
        'timestamp': datetime.now().isoformat()
    }

def safe_operation(self, operation_name, function, *args, **kwargs):
    """Execute operation with memory monitoring"""
    # Check before operation
    if not self.check_memory_limit(f"Before {operation_name}"):
        print(f"[Memory] ? Skipping {operation_name} - insufficient memory")
        return None

    try:
        # Log memory before
        self.log_memory(f"Starting {operation_name}")

        # Execute operation
        result = function(*args, **kwargs)

        # Log memory after
        self.log_memory(f"Completed {operation_name}")

        return result

    except MemoryError as e:
        print(f"[Memory] ? MEMORY ERROR during {operation_name}: {e}")
        self.force_garbage_collection()
        raise
    except Exception as e:
        print(f"[Memory] ? Error during {operation_name}: {e}")
        traceback.print_exc()
        raise

# Global memory monitor instance
memory_monitor = MemoryMonitor()

```